Capítulo I

Resumen: En este capítulo se discuten las características deseables que cualquier capa de persistencia debería proveer, de modo de no generar alteraciones y/o cambios en el modelo de objetos puro. Estas características no están relacionadas con tecnología alguna, de manera que pueden ser aplicables a cualquier alternativa de persistencia orientada a objetos.

Características deseables de la persistencia de objetos

Si se realizara una encuesta acerca de cuáles son las características deseables de las soluciones de persistencia para las aplicaciones, seguramente "performance" sería la palabra que más veces se mencionaría. Si bien es absolutamente necesario contar con una excelente (¿extrema?) performance, aún cuando no haya sido identificada como un aspecto crítico inicialmente, muchas veces se la pondera por sobre otros criterios de calidad de igual importancia.

Podría argumentarse que mantenibilidad del código, claridad del diseño, utilización de patrones (todas cualidades internas del software) no son tan importantes para el usuario (que usualmente sólo se fija en cualidades externas como el rendimiento o usabilidad); sin embargo esto no es totalmente cierto ya que las cuestiones de índole internas suelen ser la base a partir de la cual se obtienen las cualidades externas. En realidad, no es real el enfrentamiento entre "hacerlo bien" y "hacerlo rápido"; en otras palabras, es perfectamente posible conseguir una solución eficiente en términos de rendimiento, y que a la vez cumpla con todos los requisitos de un buen diseño.

Ahora bien, ¿cuáles son las cualidades deseables, más allá de la alternativa técnica concreta que se utilice? Más abajo hay una lista subjetiva de algunas de estas

características.

Performance sin compromisos

Obviamente este factor debía aparecer en la lista. La lectura que se debe hacer de su aparición en esta lista es que la solución que se implemente por ningún motivo debe poner en peligro el mejor rendimiento posible.

Es deseable que la solución posea el mejor diseño posible, así como que sea lo más mantenible y extensible posible, independientemente de la tecnología; pero de nuevo, sin que esto repercuta negativamente en el rendimiento. En conclusión, que esté bien diseñado desde el punto de vista de las técnicas de diseño y/o patrones no significa que sea ¡"lento"!

Transparencia

Esta característica establece que el diseñador/desarrollador no debería tener que estar todo el tiempo preocupándose por los aspectos ligados a la persistencia cuando esté trabajando en la lógica del negocio o dominio concreto. El escenario ideal sería que el desarrollador se enfocase solamente en los detalles propios del dominio, modelando con técnicas aplicables y reconocidas. Todo aspecto de la persistencia debería estar oculto a los ojos del desarrollador, de modo de no "desviar" no sólo su atención sino también para evitar que no se logre el mejor diseño posible en aras de cuestiones como performance (que como se mencionó en el punto anterior, es importante y a su debido momento se analizará y mejorará).

La falta de transparencia puede aparecer expresada de muchas maneras en el código, algunas veces de manera bastante obvia, y en otras ocasiones de manera muy sutil (y por lo tanto difícil de descubrir y mejorar).

Código sql

Probablemente sea muy obvio e incluso una exageración, pero está claro que en pos de obtener transparencia, sería bueno no "ver" código SQL dentro del código de los objetos de la aplicación. ¿Pero cuáles son concretamente las razones para argumentar que escribir código SQL en los objetos de dominio es "malo"?

Algunas de las razones que podemos citar son:

1. Dependencia de un motor de bases de datos en particular. Aún cuando se cuenta con un estándar SQL (SQL 92), es sabido que no todos los proveedores de software de bases de datos lo soportan o respetan totalmente. Por esta razón, el SQL de una base de datos ORACLE seguramente diferirá del específico para MySQL. Un ejemplo simple: en MySQL se puede contar una tabla denominada USER que mantenga la información de todos los usuarios de un sistema, mientras que en ORACLE, USER es una palabra reservada.

2. Dependencia de una tecnología en particular. Además del punto anterior, que pasaría si en vez de bases de datos relacionales se pretendiera utilizar una solución de bases de datos orientadas a objetos o bases de datos XML. La única alternativa sería contar con tantas implementaciones de las clases de dominio como alternativas tecnológicas de persistencia existan[1]. Cabe aclarar que se podrían aplicar técnicas de diseño y ciertos patrones de diseño para mejorar la solución, pero sólo se conseguiría minimizar el problema, no eliminarlo.

3. Cambios en una capa que afectan directamente a otra capa. La aplicación de un estándar nuevo para la nomenclatura de los elementos de una base de datos no debería afectar el código de otra capa, particularmente la que contiene la lógica de negocios. Está claro que cambiar el nombre de una columna no debería impactar en cómo se resuelve cierta lógica en los objetos de dominio. Al tener el SQL embebido en los objetos, es usual ver este problema.

[1] La idea sería tener una clase User que sepa persistirse a bases de datos relacionales. Otra clase User que sepa persistirse a bases de datos orientadas a objetos, y así siguiendo. En resumen, una “variante” de cada clase para cada tipo de base de datos.

4. Para lenguajes como JAVA, el SQL no es más que una cadena de caracteres (String), con lo cual hasta que no se lo ejecute es muy difícil saber si es válido o si contiene errores. Los cambios en el esquema de bases de datos no se ven reflejados directamente en las sentencias SQL.

Se podría escribir un capítulo entero sobre las desventajas que tiene este enfoque, pero con los puntos citados anteriormente puede comprenderse el problema en términos generales.

Dependencia de clases específicas

Hace unos años atrás[2], en un esfuerzo por lograr un estándar para las bases de datos orientadas a objetos, el OMG (Object Management Group) estableció un estándar denominado ODMG. Si bien en términos generales fue muy positivo, imponía ciertas restricciones a la hora de diseñar soluciones persistentes. Para aquellos casos en los que un objeto debería trabajar con una colección de ciertos colaboradores, el tipo de dicha colección debía cambiarse de los tipos tradicionales de JAVA (por ejemplo Collection, Vector, etc) a DVector. Esta última clase era una definición particular del estándar ODMG, la cual debía ser implementada por cada proveedor de bases de datos. La consecuencia directa de este tipo de restricciones es que uno no puede separar totalmente las capas, particularmente las de dominio y bases de datos. Ni siquiera se podrían realizar pruebas de unidad[3] ya que cada clase persistente tendría referencias a clases que requieren de un respaldo de una base de datos.

Otras soluciones incluso establecían que las clases persistentes no extendieran de Object, sino de una clase especializada PersistentObject, la cual proveía ciertas implementaciones requeridas para la persistencia.

[2] Más precisamente entre los años 1993 y 2001.

[3] Por ejemplo a través del framework JUnit.

Diseño débil o "poco" orientado a objetos

Si finalmente bajo la capa de dominio tendremos una base de datos, e incluso tendremos también algún código SQL embebido en el código, cabe preguntarse ¿por qué modelar completamente en objetos la lógica de negocios?, si al final de cuentas la resolución de los problemas en parte pasarán por consultas SQL (con sus joins, búsquedas por claves foráneas, etc.). Bajo este supuesto (mucho más habitual de lo que cabría esperar) suelen verse diseños en los cuales hay relativamente pocas clases, que incluso no tienen relaciones con las demás clases, sino que simplemente conocen los "ids" de sus colaboradores. Este es un ejemplo de un modelo débil, que más que ser un buen diseño de objetos no es más que una representación con clases de cada una de las tablas de una base de datos.

Ortogonalidad

Cuando se menciona el término ortogonalidad, hay dos semánticas posibles asociadas a la persistencia. La primera es **ortogonalidad de persistencia**, que es la expresión formal del punto anterior que planteaba la transparencia para el desarrollador respecto de los aspectos de persistencia. Más formalmente entonces, se dice que un concepto A es ortogonal a otro concepto B si al pensar en A no es necesario tener presente B; en otras palabras, una buena solución de persistencia debería permitir que el pensar una solución a un problema de un dominio fuese ortogonal a los problemas de la persistencia.

La otra acepción que se suele relacionar con ortogonalidad es **ortogonalidad de tipo**, que establece que un objeto debería ser persistente más allá de su tipo. Nuevamente, simplificando el concepto tendríamos que toda cualquier instancia de cualquier clase debería ser persistente. Es decir que no hay clases persistentes y clases no persistentes, sino que las instancias de cualquier clase se deberían poder almacenar. Sin embargo, en la práctica este punto suele materializarse a través de la declaración de las clases persistentes en archivos con meta-data (usualmente en archivos XML), ya que no es

posible que la solución de persistencia se dé cuenta "automáticamente" como persistir nuevas instancias de clases desconocidas.

POJO

El término P.O.J.O. significa **Plain Old Java Objects**[4]. Un nombre que no explica mucho ¿no? Este nombre se acuñó años atrás para referirse a las clases Java que se diseñaban y desarrollaban y que no contenían más que el código propio del dominio. Con la explicación anterior tampoco aclara mucho, ya que cabe pensar ¿qué otro código se escribiría en una clase que no fuera código relacionado con el negocio/dominio?

Bien, un poco de historia entonces para aclarar: a partir de la necesidad de poder invocar mensajes en objetos remotos (es decir objetos alojados en espacios de memoria diferentes, posiblemente incluso en máquinas distintas) Sun MicroSystems elaboró un estándar denominado **E.J.B.** (**Enterprise Java Beans**). Bajo este estándar era posible crear objetos que "publicarían" sus protocolos para que fueran invocados desde cualquier otro lugar. Una de las más importantes características de los EJB era que no eran administrados por el desarrollador, sino que por el contrario se los "instalaba" en un servidor de aplicaciones, el cual se hacía cargo de administrar el ciclo de vida de los *ejbs*, llevándolos a memoria cuando se los requería, pausándolos cuando no se los requería, etc.

Esta administración por parte del servidor, si bien resultó ser una buena idea, su implementación impuso restricciones fuertes tales como la necesidad de escribir métodos en los objetos para que fueran invocados por el servidor a su debido momento. El problema fue que muchos desarrolladores perdieron la tan ansiada transparencia, y terminaron escribiendo métodos "basura" en sus clases, ya que muchas veces no les interesaba escribir estos métodos en su totalidad, pero al utilizar el estándar EJB era lo que se debía hacer.

[4] Algo así como viejos y comunes objetos Java.

Moraleja, una característica deseable sería no tener que escribir nada más que el código propio de la aplicación, sin distraerse por otros detalles como concurrencia, distribución, persistencia, etc.

Persistencia por alcance

En la introducción hablábamos de que el concepto fundamental y guía de este libro es la **persistencia por alcance** (también se la suele mencionar como persistencia transitiva). Dicho concepto establece que todo **objeto al cual se pueda llegar a partir de un objeto ya persistente, debe ser necesariamente persistente**. Si bien es bastante simple lo que define, en realidad lo que nos interesa es como afectan las implicancias de este concepto al diseño y desarrollo de software, ya que otra forma de interpretar la persistencia por alcance es que a partir de ahora para persistir objetos es necesario solamente vincularlos con algún objeto ya persistente.

A continuación se muestra el código de la clase Person[5], la cual tiene un colaborador perteneciente a la clase Phone:

[5] Si bien soy un ferviente defensor de la utilización de nombres en los idiomas nativos de cada desarrollador, es cierto que al tener todas las palabras claves en inglés, el nombre de la clase también se ajusta mejor al inglés.

Person.java

```java
package poo;

public class Person {

    protected Phone phone;

    public Person() {
    }

    public Phone getPhone() {
        return this.phone;
    }

    public void setPhone(Phone aPhone) {
        this.phone = aPhone;
    }

}
```

Figura 1: código fuente de la clase Person

Phone.java

```java
package poo;

public class Phone {

    protected String number;

    public Phone() {
        this.setNumber("");
    }

    public Phone(String aNumber) {
        this.setNumber(aNumber);
    }

    public String getNumber() {
        return this.number;
    }

    public void setNumber(String aNumber) {
        this.number = aNumber;
    }
}
```

Figura 2: código fuente de la clase Phone

Teniendo en cuenta el código de ambas clases, y por increíble que parezca, para persistir un nuevo teléfono basta con asignárselo a la persona apropiada mediante las siguientes líneas de código (solamente la línea 12).

```
8        Person aPerson = null;
9        // se recupera a la persona apropiada de la base de datos;
10
11       Phone aPhone = new Phone("123 456 789");
12       aPerson.setPhone(aPhone);
```

Figura 3: Código necesario para persistir un teléfono

Realmente increíble ¿no?

Las anteriores líneas de código merecen una explicación más detallada para su mejor compresión:

1. Las líneas 8 y 9 de la figura 3 en las que se recupera una persona por el momento no son más que un pseudo-código ya que aquí es justamente donde se debe interactuar con el repositorio de objetos a fin de recuperar una instancia ya persistente de la clase Person. Una vez que la instancia se recupera, todo objeto que sea vinculado a ella deberá ser a su vez persistente una vez que se finalice exitosamente la ejecución del método que contiene estas líneas.

2. La línea 11 muestra que no es necesario escribir ningún código especial para que las nuevas instancias de teléfono se almacenen. Los objetos persistentes se crean a través de sus constructores habituales y cuando se los vincule a otro objeto ya existente se persistirán.

3. La línea 12 es la que realmente hace "la magia", pero sin embargo en realidad lo único que ejecuta es el setter de la variable de instancia *phone* (revisar el código mostrado previamente para asegurarse que así sea).

Si revisamos el código de ambas clases se podrá observar que en ninguno de los dos casos tenemos código de base de datos ni tuvimos que alterar nuestro diseño natural de la relación entre las personas y sus teléfonos debido al conocimiento que tenemos de

que finalmente ambas instancias serán persistentes. Este es entonces un buen ejemplo del concepto de transparencia para el desarrollador al que nos referíamos al principio de este capítulo.

Por supuesto que este no es todo el código necesario en una aplicación real, pero sirve como una muestra de lo que implica la **persistencia por alcance** desde el punto de vista del diseño y su posterior implementación.

¿Qué pasaría si ahora nos interesara tener una persona que a su vez tiene un conjunto de teléfonos y no uno sólo como en el ejemplo anterior? En este caso, la clase Person sería la siguiente

Person.java

```
1  package poo;
2
3  import java.util.Collection;
4  import java.util.HashSet;
5
6  public class Person {
7
8      protected Collection<Phone> phones;
9
10     public Person() {
11         this.setPhones(new HashSet<Phone>());
12     }
13
14     public Collection<Phone> getPhones() {
15         return this.phones;
16     }
17
18     public void setPhones(Collection<Phone> somePhones) {
19         this.phones = somePhones;
20     }
21
22     public void addPhone(Phone aPhone) {
23         this.getPhones().add(aPhone);
24     }
25
26 }
```

Figura 4: Código de la clase Person con una colección de teléfonos

Como se puede observar, nuevamente no existe código que de alguna manera esté relacionado con una base de datos. El código de la clase Person continúa siendo puramente "de dominio".

Para persistir una nueva instancia de teléfono bastaría con ejecutar el método **addPhone()** de la clase Person con la instancia de teléfono y listo!

Aquí en este punto es donde es posible vislumbrar los cambios en el diseño, implementación e incluso a nivel de arquitectura de software que tiene la **persistencia por alcance.**

Operaciones C.R.U.D.

En la bibliografía disponible actualmente suele identificarse un conjunto de operaciones necesarias e indispensables para toda aplicación orientada a objetos que deba persistir su información. Dicho conjunto es conocido como operaciones C.R.U.D. (por sus siglas en inglés):

1. **C** (Create): esta operación es la encargada de crear nueva información persistente. A partir del conocimiento de la **persistencia por alcance** esta operación no debería existir como una operación en sí misma, sino que está representada por el mismo código de dominio que crea y asigna las instancias a los objetos que correspondan. En otras palabras, no debería existir una operación "**create**" o "**save**" para almacenar instancias en el repositorio.

2. **R** (Retrieve, Read): esta operación es la encargada de recuperar la información del modelo. Probablemente si se tuviera que optimizar la performance de la aplicación, esta operación sería el principal punto para hacerlo. El modelo es perfectamente capaz de resolverlo, pero como mencionamos más arriba, esta es una buena oportunidad de optimizar la performance.

3. **U** (Update): en una aplicación tradicional, para actualizar un objeto basta con enviar un mensaje (setter) con la información nueva a actualizar. En un modelo persistente, no debería ser diferente si se trata de obtener transparencia. En otras palabras, no es necesario tener operaciones "**update**" diferenciadas para actualizar la información.

4. **D** (Delete): para borrar un objeto del modelo lo único que debería hacerse es "desvincularlo" apropiadamente de sus colaboradores. Nuevamente, no es necesario tener un "**delete**" específico en el modelo.

Más adelante, cuando tratemos el tema del patrón DAO volveremos sobre las operaciones C.R.U.D. para demostrar que dicho patrón de diseño trae más problemas y falencias en el diseño que beneficios.

Capítulo II

Resumen: En este capítulo se repasan conceptos esenciales de ambientes concurrentes y otros conceptos fundamentales como transacciones, esquemas de bloqueo y versionamiento. Dichos conceptos serán siempre abordados desde la problemática que generan en el diseño y desarrollo orientado a objetos que requiere de persistencia en bases de datos.

Propiedades A.C.I.D.

Estas propiedades establecen un marco de trabajo y restricciones que deben ser cumplidas en un ambiente concurrente y multiusuario a fin de mantener la información accesible, confiable, siempre en forma compartida entre todos los usuarios. Cada una de las propiedades está enfocada en un aspecto en particular:

1. A (de Atomicity en inglés o Atomicidad): al interactuar con un repositorio de información (por ejemplo una base de datos relacional) usualmente se invocan series de operaciones más que una sola operación en particular. En este escenario, un requerimiento importante a cumplir es que todo el conjunto de operaciones enviadas a la base de datos se deben ejecutar exitosamente como una sola unidad. En caso de que alguna de estas operaciones falle, todos los cambios producidos por las demás operaciones se deben deshacer a fin de mantener la consistencia de la información. La atomicidad justamente es la propiedad que resguarda este punto al establecer que se ejecuta todo como una unidad indivisible o no se ejecuta nada.

2. C (de Consistency en inglés o Consistencia): esta propiedad es la encargada de mantener la coherencia de la información, asegurando que la información nunca pase a un estado incoherente. Todo conjunto de operaciones siempre debe pasar

la información de un estado coherente a otro, a fin de mantener la confianza en el repositorio.

3. I (de Isolation en inglés o Aislamiento): en un ambiente concurrente y multiusuario, es de esperar que cada usuario envíe una serie de operaciones a la base de datos. En este contexto, no sería adecuado que cada operación de cada usuario se ejecute sobre el ambiente compartido en forma libre ya que podría estar afectando el conjunto de datos que otra operación de otro usuario podría estar leyendo o incluso alterando en forma simultánea. Para evitar este problema, cada conjunto de operaciones se debe ejecutar en forma aislada respecto de los otros conjuntos de operaciones. Una vez que todo el conjunto de operaciones se haya ejecutado en forma completa y satisfactoria, recién en ese momento los cambios serán visibles para los demás usuarios.

4. D (de Durability en inglés o Durabilidad): finalmente esta propiedad establece que un cambio realizado satisfactoriamente a la información mantenida por un repositorio, y que haya cumplido satisfactoriamente las 3 propiedades anteriores, deberá ser mantenido por dicho repositorio soportando cualquier situación anormal, por ejemplo fallas. A partir de la ejecución satisfactoria de un cambio, el nuevo estado de la información se debe mantener como base para cualquier otro cambio que se realice en el futuro.

La forma tradicional de mantener las 4 propiedades es a través de la demarcación de las operaciones utilizando transacciones. Utilizando éstas últimas, el desarrollador irá estableciendo el conjunto de operaciones que deben ser ejecutadas como una sola unidad (atomicidad), las cuales deberán dejar la información consistente (consistencia), siempre en un ambiente aislado (aislamiento) y en forma durable (durabilidad).

En la siguiente sección se discutirán más en detalle las transacciones, presentando las características especiales que hay que tener presente al momento de trabajar con éstas en un ambiente orientado a objetos.

Transacciones

Como mencioné en el punto anterior, las transacciones suelen ser el medio más adecuado para mantener y resguardar la información a través del aseguramiento de las 4 propiedades A.C.I.D.

También se mencionó que el desarrollador es el encargado de demarcar el conjunto de operaciones que deben ser ejecutadas como una unidad. Si bien esto es un tema ya conocido, el problema al que nos enfrentamos es que este conjunto de operaciones no es precisamente un conjunto de sentencias SQL, sino que si estamos en un ambiente orientado a objetos, por lo que estas "sentencias" serán en realidad mensajes enviados a distintos objetos.

Para tratar de aclarar el punto anterior, tenemos que analizar varios puntos relacionados con las transacciones:

1. Si consideramos que estamos trabajando en un ambiente orientado a objetos puro, es de esperar que todo sea un objeto, en particular las transacciones serán objetos. Esto significa que se le podrán enviar mensajes a las transacciones, por ejemplo:

    a. begin() que demarcará el inicio de la transacción.

    b. commit() que notificará a la transacción que trate de bajar a la base los cambios realizados mediante el envío de mensajes a los objetos.

    c. rollback() que indica a la transacción que deshaga los cambios realizados a los objetos.

2. Mencionamos más arriba que es el desarrollador el que debe demarcar en el código el inicio y fin de las transacciones, por lo cual estamos poniendo en peligro el concepto de transparencia de persistencia establecido en el capítulo I. La figura 5 muestra un ejemplo de este punto.

ExplicitTransactions.java

```
1  package poo;
2
3  public class ExplicitTransactions {
4
5      public void aMethod() {
6
7          Transaction tx=//se crea una transacción
8
9          tx.begin();
10         Person aPerson=//se recupera una persona de la
11                        //base de datos
12         aPerson.setPhone(new Phone("123 456 789"));
13
14         tx.commit();
15
16
17     }
18
19 }
20
```

Figura 5: Demarcación explícita de transacciones

3. La figura anterior no sólo muestra la demarcación de transacciones explícita (líneas 7, 9 y 14), sino que también presenta un ejemplo de cómo se envían mensajes a los objetos involucrados en la resolución de cierta lógica de negocios. El problema aquí es que la solución de persistencia que se esté utilizando debe llevar el registro de los cambios que se van realizando a cada uno de los objetos involucrados, tanto a fin poder bajar los cambios como para desestimarlos y volver al estado anterior a cada uno de los objetos afectados.

4. Si se recuerda el concepto de **persistencia por alcance**, al realizar el commit de la transacción la solución de persistencia aplicará dicho concepto e irá recorriendo todos los objetos modificados dentro de la transacción para analizar si corresponde actualizarlo (en caso de que ya sea un objeto persistente) o si por el contrario se trata de un objeto volátil (y por ende se lo debe persistir).

5. Un punto que es necesario aclarar es el relacionado con el "momento" en el cual se crean las nuevas instancias que luego se persistirán. En el ejemplo de la figura 5, en la línea 12 se crea una nueva instancia de clase Phone, pero no hay ninguna restricción respecto de que esto siempre tenga que ser así. Se podría haber pasado por parámetro al método del ejemplo un objeto teléfono para no tener que crearlo.

Lectura de la información

Por lo dicho anteriormente, salvo que se cuente con herramientas adicionales[6], el código de nuestro diseño tendrá llamadas explícitas a las transacciones. Un punto importante que hay que tener en cuenta es cómo las transacciones ayudan a mantener las propiedades A.C.I.D. y cómo terminan afectando la manera de pensar a la que estamos habituados. Para ello podemos analizar en detalle el código de la siguiente figura.

```
5   public void aMethod() {
6       Phone aPhone = new Phone("123 456 789");
7       Transaction tx = // se crea una transacción
8
9       tx.begin();
10      Person aPerson = // se recupera una persona de la
11      // base de datos
12      aPerson.setPhone(aPhone);
13
14      tx.commit();
15
16      System.out.println("Teléfono " + aPhone.getNumber());
17  }
```

Figura 6: Alcance de las variables afectadas por las transacciones

[6] Existen herramientas y frameworks que solucionan en parte este problema, y de hecho este libro contiene un capítulo sobre este punto, pero por el momento se asume que no se cuenta con dichas herramientas por lo que agregar transacciones inevitablemente "ensucia" el código.

Analizando en detalle la figura 6, puede verse que el código es bastante similar al de la figura 5, excepto que la instancia de teléfono es creada en forma separada (línea 6) y luego asignada a la persona (línea 12). La demarcación de las transacciones establece que la asignación del teléfono a la persona debe hacerse protegiendo las propiedades A.C.I.D. por lo que al finalizar la transacción (línea 14) se persistirá la nueva instancia de teléfono por persistencia por alcance.

Hasta aquí no hay mayores novedades. Los problemas empiezan cuando se trata de ejecutar la línea 16. En dicha línea básicamente lo que se espera obtener es una impresión en la consola del número de teléfono de la instancia persistida. Si bien no es "nada de otro mundo", el código de la figura 5 lamentablemente falla[7]. ¿Pero por qué falla si el alcance de la variable que referencia al teléfono llega hasta la línea 17 inclusive?

La razón de esta falla es que con el objetivo de mantener las propiedades A.C.I.D., no está permitido leer información relacionada con la base de datos fuera de las transacciones (recordar que al persistir la información, se está compartiendo la información en un ambiente concurrente multi-usuario). Luego de la línea 14 otro usuario podría estar modificando la información y no se estaría accediendo a dicha información actualizada si no se accediera dentro de un marco transaccional.

Semejante restricción implica cambios profundos en la forma de diseñar y codificar, ya que la intención de persistir información en una base de datos es poder consumirla más tarde, pero ¿cómo se puede acceder a dicha información si cada vez que cerremos las transacciones perderemos la posibilidad de acceder a la misma?

Por suerte existen varias alternativas para salvar este problema:

1. Abrir la transacción el mayor tiempo posible, de modo que toda la recuperación de la información siempre esté dentro del marco de las transacciones. Es obvio que esta alternativa no es válida en una aplicación real, pues implica que toda interacción con el usuario a través de una interfaz gráfica deberá ser dentro de

[7] Por cuestiones de optimización de rendimiento, en realidad el código tal cual está escrito no falla. Al acceder a atributos básicos usualmente no se dan errores ya que se cuenta con la información "en memoria". En cambio si lo que se está accediendo es un colaborador (instancia de otra clase) el código en cuestión sí dará una excepción.

transacciones y por ende las mismas deberían tener una duración extremadamente grande (estamos hablando de segundos o incluso minutos para que el usuario tome decisiones que permitan cerrar las transacciones). Durante todo el tiempo que el usuario tiene abierta una transacción se podrían estar generando cambios en la base de datos que dicho usuario no estaría viendo y que recién al tratar de hacer **commit** con su transacción se harían visibles (posiblemente generando excepciones debido a la "vejez" o invalidez de sus datos). Esta alternativa se menciona simplemente por una cuestión de completitud.

2. Utilizar los objetos desasociados[8]. Este nombre se genera a raíz de que esta alternativa plantea la posibilidad de "desasociar" o "desligar" los objetos recuperados de las transacciones en las cuales se los ha obtenido. En otras palabras, según esta alternativa es posible ejecutar satisfactoriamente el código de la figura 6 sin que se produzca un error en la línea 16. Si bien es una alternativa válida, por cuestiones más avanzadas para este capítulo introductorio (como optimización de performance), su aplicación no suele estar exenta de problemas. En capítulos posteriores nos referiremos a esta alternativa con más detalle.

3. Usar DTOs (Data Transfer Objects): como su nombre lo indica, esta alternativa se basa en la utilización de objetos que sirven solamente para transferir datos o información. Básicamente esta alternativa plantea una solución muy simple al problema de la lectura de la información fuera de las transacciones. Dicha solución establece la carga de la información relevante de los objetos recuperados dentro de las transacciones en objetos volátiles que puedan ser leídos fuera del contexto transaccional. La diferencia radical con la alternativa de los "detached objetcs" es que no se está desasociando a los objetos recuperados de sus respectivas transacciones, sino que se están creando "copias" de los mismos. Al ser copias no estarán vinculados con transacción alguna, por lo que se podrá acceder a su información aún fuera de las transacciones. Las siguientes figuras muestran más claramente este punto y permiten dar un primer

[8] El término en inglés es 'Detached Objects".

vistazo al patrón de diseño DTO (el capítulo VI contiene una explicación detallada de este patrón, sus beneficios y contras).

PhoneDTO.java

```java
1  package poo;
2
3  public class PhoneDTO {
4
5      protected String number;
6
7      public PhoneDTO(Phone aPhone) {
8          this.setNumber(aPhone.getNumber());
9      }
10
11     public String getNumber() {
12         return this.number;
13     }
14
15     public void setNumber(String aNumber) {
16         this.number = aNumber;
17     }
18
19 }
20
```

Figura 7: un DTO para una instancia de la clase Teléfono.

```java
5      public void aMethod() {
6          PhoneDTO aDTO;
7          Transaction tx = // se crea una transacción
8
9          tx.begin();
10         Phone aPhone = // se recupera un teléfono de la
11         // base de datos
12         aDTO = new PhoneDTO(aPhone);
13
14         tx.commit();
15
16         System.out.println("Teléfono " + aDTO.getNumber());
17     }
```

Figura 8: código de ejemplo utilizando los DTOs.

Nuevamente vale realizar algunas aclaraciones sobre las dos figuras anteriores:

1. El código de la clase PhoneDTO (figura 7) es muy similar al código de la clase Phone (figura 2). Sin embargo, la intención de clase PhoneDTO es simplemente transferir información, a través de sus atributos y sus correspondientes getters y setters. En la figura 2 no se muestra, pero es de esperar que la clase Phone contenga lógica de dominio/negocio que amerite la existencia de dicha clase. Otro punto a notar es que el modelo (Phone) no conoce al DTO (PhoneDTO) sino que el conocimiento es a la inversa. Más adelante volveremos sobre este punto a fin de comentar los beneficios de este enfoque.

2. El DTO no contiene absolutamente ninguna lógica, está compuesto únicamente por getters y setters.

3. Los DTOs refuerzan la separación en capas, ya que no presentan más comportamiento que sus getters y setters, por lo que no es posible invocar lógica de negocios sobre ellos. A diferencia de los DTOs, una desventaja importante de un Detached Object es que contiene toda la lógica de negocio, con lo cual al enviarlo por ejemplo al cliente, éste podría equivocarse e invocar cierta lógica en forma indebida.

Finalmente, considerando nuevamente la figura 6 ¿porqué podría fallar el código? La respuesta a esta pregunta suele desconcertar a los desarrolladores que no están habituados a trabajar en ambientes multi-usuario y concurrentes. Además de todos los problemas tradicionales, debemos agregar un nuevo punto que podría generar problemas. Dicho punto se encuentra en la línea 14 (la línea del **commit**). En esta línea es donde la solución de persistencia que se esté utilizando deberá controlar que la información pueda enviarse a la base de datos manteniendo las propiedades A.C.I.D. y es justamente por dicha razón que si la solución de persistencia nota que el usuario está tratando de agregar un nuevo teléfono a una persona que ha sido borrada de la base por otra transacción se levantará una excepción para señalar esta situación irregular.

Como conclusión, cuando se está realizando un debugging de la aplicación, pasar exitosamente las líneas correspondientes a la lógica de negocios no significa que ese código es seguro, ya que finalmente se evaluarán todos los cambios realizados a los

objetos cuando se realice el **commit** y podrían entonces en dicha línea levantarse excepciones no esperadas.

Esquema de lockeo

En la sección anterior se mencionó que al tratar de realizar un commit podría darse un error debido a una edición de los objetos por parte de otro usuario. En realidad, esto podría ocurrir siempre y cuando se permita que dos usuarios puedan acceder y modificar los mismos objetos. De hecho, esta es una de las dos alternativas existentes:

1. Optimista: esta estrategia supone que no existirán conflictos de ediciones concurrentes de la misma información, por diferentes usuarios. A partir de esta suposición se permite el acceso a los usuarios a todos los objetos del repositorio.

   La clara ventaja de esta alternativa es que ningún usuario queda bloqueado esperando el acceso a otro objeto porque algún otro usuario ya lo tiene "tomado". Todo usuario puede acceder a prácticamente todo objeto.

   Por el lado de las desventajas tenemos que mencionar los problemas que se dan cuando se detecta un conflicto de edición concurrente. En este caso, probablemente una de las dos ediciones concurrentes esté destinada a fallar.

2. Pesimista: esta alternativa supone que seguramente existirán conflictos entre los diferentes usuarios, por lo que se van bloqueando los objetos a medida que son tomados por los usuarios. De esta manera en todo momento solamente un usuario podrá modificar un objeto dado.

   Las desventajas de esta alternativa son dos: por un lado podrían darse problemas de "inanición" para un usuario que está esperando un objeto que nunca es liberado por otro usuario; la segunda desventaja tiene que ver con la degradación muy importante de la performance, ya que todos los accesos a los objetos se deben serializar.

La siguiente sección presenta el concepto de versionamiento, el cual suele utilizarse como mecanismo de control para los ambientes con esquema de lockeo optimista.

Versionamiento

Este concepto suele generar confusión debido a los diferentes significados que se le asignan dependiendo del autor. Aquí presentaremos los dos significados más relevantes, en particular enfocándonos sobre el versionamiento de instancias que permite el funcionamiento de los esquemas optimistas.

Versionamiento de clases

El versionamiento de clases define el conjunto de cambios al esquema de clases del diseño que se soportará cuando se implemente una solución de persistencia. En otras palabras, define el conjunto de cambios que se podrán realizar a las clases una vez que tengan instancias persistidas, así como qué acciones deben ser tomadas para mantener la consistencia de la información. A grandes rasgos, los cambios posibles se dividen en dos grandes grupos:

1. Cambios suaves

Estos son cambios que pueden demorar su aplicación y/o hacerse bajo demanda. Ejemplo típico de este cambio es agregar una nueva variable de instancia a una clase con instancias ya persistidas. ¿qué acciones deberían realizar inmediatamente en las instancias persistentes luego de agregar la nueva variable de instancia en la clase? En realidad ninguna, ya que podría esperarse a ir recuperando cada instancia bajo demanda y asignándole en ese momento un valor por defecto (recordar que en Java cada tipo tiene su valor por defecto).

2. Cambios duros

A diferencia de los cambios suaves, los duros establecen que se debe detener el acceso de los usuarios a la base de datos y realizar inmediatamente el cambio a las instancias. Un ejemplo de este caso es dividir una clase en una jerarquía con clases abstractas y concretas. En este caso no queda otra alternativa que recorrer el conjunto de instancias y proceder a asignarle su nueva "clase".

Versionamiento de instancias[9]

El versionamiento de instancias por su parte tiene como objetivo asegurar que las ediciones concurrentes sean llevadas a cabo de manera que no afecten la información llevándola a estados inconsistentes.

En este punto nuevamente tenemos dos alternativas: la primera más compleja pero a su vez más abarcativa, mientras que la segunda es muy simple de implementar pero un poco "restringida" en su alcance.

1. Versionamiento integral

El esquema de versionamiento "integral" permite contar con objetos y versiones de éstos. Cuando se crea un nuevo objeto, se considera que éste es el original. Luego, el usuario tiene herramientas para pedir la creación de "copias" o versiones del objeto original. Cada objeto puede contar con un grafo dirigido de versiones. En este esquema hay que resaltar que los demás objetos pueden elegir apuntar a cualquier versión de otro objeto (tanto el original como cualquier versión derivada). Como puede observarse, este esquema es complejo de implementar, pero a la vez muy potente.

2. Versionamiento simple

El esquema es realmente básico. Incluso tampoco es algo novedoso, ya que se lo conoce desde hace mucho tiempo. Lo que se plantea es agregar una nueva variable de instancia a las clases que serán persistentes para almacenar un número que indica a qué versión corresponde una instancia determinada. Cada vez que se modifique exitosamente (es

[9] El esquema que se menciona aquí es sumamente simple. Otras alternativas más eficientes y complejas pueden ser implementadas. Este esquema simple se comenta con el fin de demostrar cuán fácil puede ser llevar el control de la versión de un objeto en particular. Un punto no cubierto por esta alternativa es el versionamiento de todo un grafo de objetos.

decir se pueda realizar efectivamente el **commit** de la transacción asociada) una instancia, este número se incrementará automáticamente.

Este esquema sigue los siguientes pasos entonces:

I. Cualquier usuario recupera un objeto de la base de datos, con un número de versión dado. Dicho número de versión es mantenido automáticamente por la solución de persistencia que se esté utilizando.

II. Cada vez que un usuario edita la información de sus objetos y puede realizar exitosamente un **commit** de la transacción, dicho número de versión se incrementa automáticamente.

III. El resto de los usuarios que también habían leído la misma instancia con el mismo número de versión, cuando quieran persistir sus cambios lo harán con un número de versión "viejo" por lo que la transacción fallará. En este caso el código debe estar preparado para este "incidente" ya que no es la lógica de negocios la que falló, sino que ocurrió un problema de concurrencia.

Como puede verse en la descripción de pasos anterior, no es más que un esquema típico de "timestamp", pero aplicado a objetos en esta ocasión y con un control automático por parte de la solución de persistencia utilizada.

Capítulo IV

Resumen: Como se comentó al inicio de este libro, la situación más probable al desarrollar un nuevo sistema de software es que se utilicen dos paradigmas distintos: el orientado a objetos para el diseño y construcción de la aplicación, y el relacional para la persistencia de la información. Por esta razón en este capítulo se presentan los principales conceptos del mapeo objeto/relacional, primero en forma y luego se comenta brevemente el producto Hibernate a modo de ejemplo de implementación de los conceptos previos.

Diferencia de impedancia

El primer punto importante que se debe considerar es la diferencia existente entre los dos paradigmas que se están haciendo coexistir. Por un lado tenemos el paradigma relaciona, basado fuertemente en nociones matemáticas; por otro lado tenemos al paradigma orientado a objetos, el cual tiene sus fundamentos en la Ingeniería de Software. Las diferencias resultantes entre los dos paradigmas fueron bautizadas en los ´90 como “diferencia de impedancia”.

Al trabajar con objetos, mapeándolos a una base de datos relacional, es necesario conocer y tener bien en claro dichas diferencias, a fin de poder diseñar y desarrollar aplicaciones que se integren de la mejor manera posible.

Como ejemplo de las diferencias entre ambos paradigmas, suele considerarse el mecanismo de acceso a la información: en objetos, la forma tradicional de acceder a la información es a través de la navegación de los colaboradores, mientras que en relacional la forma recomendada es a través de *joins* entre diferentes tablas. Cierta no es el mismo mecanismo en ambos paradigmas.

Existen otras diferencias más sutiles y a la vez básicas que conviene tener en mente:

- Datos y lógica separados: en el paradigma orientado a objetos la información y la lógica encargada de operar sobre dicha información se encuentran en un solo lugar: los objetos. En cambio en el paradigma relacional, la información se encuentra en tablas, mientras que la lógica se modela en procedimientos, funciones o "triggers". En el caso del relacional, el modo de trabajo referencia mucho al paradigma procedural.

- Objeto que representa el sistema[18]: una práctica común en el diseño orientado a objetos es modelar con un objeto en sí mismo al sistema que se está modelando. En otras palabras, si estoy modelando un sistema para un banco, probablemente la primera clase que aparezca en mi diseño sea Banco. De la misma manera, al modelar un sistema para un aeropuerto, la clase Aeropuerto seguramente aparecerá. El objetivo de dicha clase es representar al sistema bajo diseño, siendo generalmente responsable por ejemplo de las colecciones de objetos importantes, por ejemplo ¿dónde se definiría la lógica de administración de clientes de un banco si no es en la clase Banco? Ahora bien, en el paradigma relacional si uno intenta crear una entidad Banco en un diagrama Entidad-Relación, lo más probable es que termine recursando la materia de bases de datos[19].

- Tipos de datos: otro punto con notables diferencias es el relacionado con los tipos de datos. En Java por ejemplo tenemos el tipo String, mientras que en Oracle existe el tipo Varchar2. No sólo no son el mismo tipo, sino que incluso ambos tipos cuentan con funcionalidad y restricciones diferentes (el tipo Varchar2 tiene un tamaño máximo de 2500 caracteres).

- Colecciones en objetos y relacional: en el paradigma orientado a objetos se cuenta con colecciones, las cuales modelan distintas situaciones, como por ejemplo conjuntos (Set) que no permiten elementos repetidos. En cambio en el paradigma relacional no sólo no existe el concepto de colección, sino que el

[18] En el capítulo dedicado a los patrones de diseño en persistencia se presenta un patrón que establece justamente este punto.

[19] El motivo que se esgrime en relacional es que no tiene sentido modelar una entidad Banco, que se sabe que luego al transformarse en tabla contendrá únicamente una sola tupla, que difícilmente varíe. Notar que lo mismo pasa en el paradigma orientado a objetos, salvo que en este caso la única instancia tiene lógica asociada.

único mecanismo para representarlo (las relaciones) no tienen toda la semántica asociada. Como muestra de esto último para modelar un conjunto es necesario definir restricciones o "*constraints*" que velan por la ausencia de tuplas con ciertos atributos repetidos.

- Jerarquías en objetos y en relacional: según lo que se vio en el capítulo anterior, al realizar una consulta con una superclase como destino en OQL uno puede obtener todas las instancias, tanto de la superclase como de todas las subclases de la misma. En cambio, al tener una jerarquía en el modelo relacional, no sólo es un concepto teórico que no tiene correlato en la implementación, sino que además en el SQL no es posible realizar una consulta a la tabla que contiene las tuplas de la super-entidad y que automáticamente se incluya a las tuplas de las entidades hijas.

- Identificación de los objetos y de las tuplas: en el modelo relacional se enseña que cada entidad tiene su identidad definida en base a sus atributos (por ejemplo el dni+nombre+apellido define a una persona). En cambio en objetos la identidad de un objeto queda definida por su OID, no siendo editable éste último por el usuario o desarrollador. Incluso, el OID ayuda a identificar unívocamente a una instancia en todo el sistema, mientras que en relacional el identificador suele restringirse a la tabla a la que pertenece, pudiendo repetirse el mismo identificador para diferentes tablas. Finalmente otro punto a señalar es que en relacional es posible identificar tuplas en base a identificadores de otras tuplas (claves foráneas). En objetos esto último no sería posible ya que se estaría ante una flagrante violación del encapsulamiento.

Características deseables del mapeo

Aún teniendo que enfrentarse con los problemas surgidos a partir de la diferencia de impedancia antes comentada, si se intentará construir una aplicación bajo el paradigma

de objetos con soporte relacional, ¿cuáles serían las características deseables de la función de mapeo entre ambos paradigmas?

Entre dichas características seguramente tendríamos:

- Idempotencia: como se mencionó en un capítulo previo, la idempotencia permite aplicar infitas veces una función, sin que ésta altere el conjunto de datos sobre los que fue aplicada, dando como resultado también todas las veces el mismo resultado. Esto es sumamente importante al momento de mapear objetos a una base de datos. El mapeo no debe alterar nunca los objetos que está persistiendo, así como también siempre debe mapearlos de la misma manera.

- Transparencia para el sistema O.O. de los detalles de la persistencia; es decir, que el desarrollador de la aplicación O.O. no debería preocuparse (por lo menos no directamente) de los detalles de performance, de representación de la información, de cuantas tablas se están consultando, ni de las transacciones y otros conceptos propios de las bases de datos relacionales.

- Control de todas las propiedades A.C.I.D. A modo de repaso, estas propiedades establecen que un conjunto de operaciones debe ser ejecutado como un todo (A, de atomicity) y en caso de no poder ejecutar alguna operación todo el conjunto de operaciones tiene que ser deshecho; el estado en el que se deja la base de datos debe ser consistente, nunca en un estado indefinido (C, de consistency); todo cambio que se haga debe ser en absoluta independencia de los cambios que otros usuarios pudieran estar realizando al mismo tiempo (I, de isolation) y finalmente que todo cambio que llegue a la base debe ser durable en el tiempo (D, de durability).

- Performance sin compromisos. Si el desarrollador de la aplicación OO tiene que tomar decisiones respecto a cómo implementar una solución teniendo en cuenta aspectos de performance por sobre otros aspectos como por ejemplo patrones de diseño, entonces no tendríamos transparencia y por lo tanto no sería una situación buena.

- Integración fácil y sin limitaciones. Si los tipos de datos a ser utilizados están restringidos por el producto que se encargue del mapeo, entonces el diseñador

no es libre de plantear la mejor solución desde el punto de vista del diseño, por lo que no se cuenta ni con transparencia ni con independencia de la base de datos.

- Meta-data. Nuevamente persiguiendo la transparencia y la mantenibilidad de la aplicación, todos los aspectos relacionados con la persistencia deberían estar lo más separados posible del código en sí. Tener la meta-data requerida para el mapeo en archivos externos es una alternativa que permite variar el mapeo sin tener que alterar el código fuente.

¿Qué se mapea y que no?

Considerando que el mapeo no es más que una función que permite traducir objetos desde y hacia una base de datos relacional, y que en definitiva lo que se está haciendo es vinculando dos paradigmas distintos (el relacional y el orientado a objetos), cabe entonces preguntarse ¿cuáles de los elementos de cada uno es posible mapear al otro?

En objetos básicamente se cuenta con dos grandes "partes": por un lado tenemos el comportamiento (lo principal) y por otro lado la estructura (funcional al comportamiento, y por lo tanto secundaria). Obviamente tanto el comportamiento como la estructura se podrían mapear, sin embargo el mapeo de cada uno de estas partes tiene ventajas y desventajas. Si tenemos en cuenta que lo que siempre se trata de evitar es repetir lógica en distintos lugares, debería quedar claro que no tiene muchas ventajas prácticas mapear el comportamiento. Si hay cuestiones de performance de por medio, puede ser necesario tener el comportamiento en la base de datos[20], pero entonces en este caso no se tendría el mismo comportamiento replicado en los objetos.

El caso de la estructura en cambio es diferente, ya que la estructura tiene como fin representar la información, la cual sí debe ser persistida en la base de datos ya que si no lo hace será siempre de carácter volátil. Es cierto que en un momento dado habrá

[20] Por ejemplo a través de funciones o stored procedures en PLSQL.

información replicada, pero esto no representa un problema ya que en memoria siempre habrá copias locales y restringidas de la información consistente almacenada en la base de datos.

De los dos párrafos anteriores entonces se puede concluir que la función de mapeo suele tener más importancia para transformar la estructura de objetos en una estructura de tablas y tuplas más que para transformar el comportamiento.

Mapeo de clases

El escenario más simple es tener que mapear la estructura de una sola clase. En este caso, dicha clase estará compuesta solamente por atributos simples (recordar que se trata de un escenario con una sola clase). Dichos atributos probablemente correspondan con tipos simples como String, boolean, etc. En este caso la función de mapeo establece que a cada atributo simple le corresponderá una columna de una tabla y a cada instancia de dicha clase le corresponderá una nueva tupla de la tabla resultante.

La figura 11 muestra este escenario.

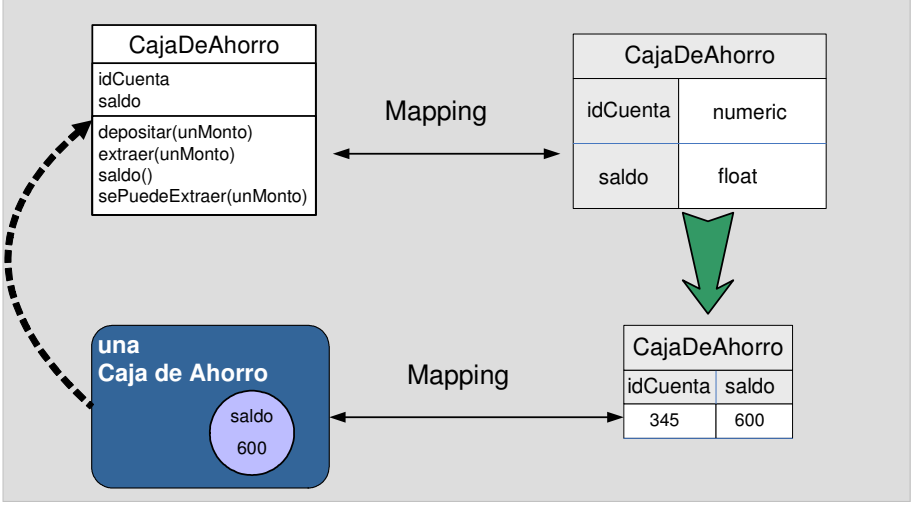

Figura 11: Mapeo de una clase a una tabla.

En conclusión se verifican las siguientes "igualdades"[21]

Clase = Tabla

Instancia = Tupla

Atributo = Columna

Notar que en el mapeo realizado en la figura 11 no contamos con una transparencia total, ya que hemos tenido que agregar una columna en la tabla para almacenar el identificador de la tupla (*pk*[22]). Luego, este atributo tiene que ser reflejado también en la estructura de la clase (atributo *idCuenta*).

Estrategias de mapeo de jerarquías

En cualquier diseño de un sistema real es muy probable que no existan clases "sueltas", sino que seguramente estarán organizadas algunas veces dentro de jerarquías de clases. En esta situación es necesario analizar las 3 estrategias de mapeo de jerarquías, ya que cada una tiene ventajas y desventajas que deben ser conocidas y comprendidas para poder tomar una decisión en cada oportunidad que se presente.

Para todos los ejemplos tomaremos la misma jerarquía de ejemplo, sobre la que se irán comentando los detalles de cada estrategia de mapeo y presentando pros y contras de cada alternativa.

La figura 12 presenta la jerarquía de clases de ejemplo.

[21] Para evitar polémicas, estos conceptos NO son "iguales" en términos de su significado, sino que son igualados a partir de la aplicación de la función de mapeo.

[22] Pk=Primary Key. Clave primaria que identifica unívocamente a una tupla dentro de la tabla.

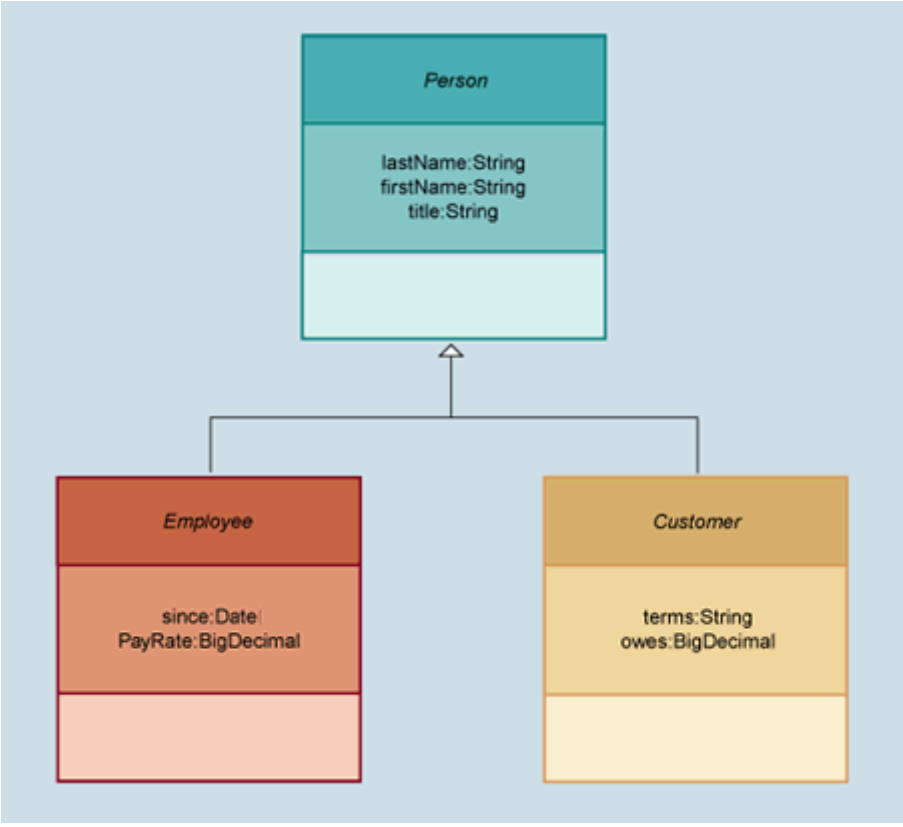

Figura 12: Jerarquía de clases.

Mapeo de la jerarquía a una sola tabla

En esta alternativa todas las clases pertenecientes a una jerarquía (incluso las clases abstractas) se mapean todas "juntas" a una sola tabla destino, que contendrá tantas columnas como atributos distintos existan en toda la jerarquía.

De acuerdo con lo anterior, para la jerarquía de personas, en esta alternativa tendríamos la tabla de la figura 13

Person
OID name phoneNumber customerNumber preferences startDate objectType

Figura 13: Tabla resultante al mapear toda la jerarquía a una sola tabla

Como puede verse en la figura 13, a cada uno de los atributos de las diferentes clases le corresponde una columna propia para mantener dicha información.

A. Ventajas de esta solución

1. Toda la información se encuentra en una sola tabla, por lo que las consultas que deban recorrer todos los elementos de una jerarquía (por ejemplo *"recuperar todas las personas que…."*) serán extremadamente rápidas ya que todas las tuplas están en la misma tabla y por lo tanto no es necesario ejecutar **joins**.

2. Cuando se realiza un cambio en la estructura de la clase al tope de la jerarquía y se quiere propagar este cambio a las tablas involucradas en el mapeo, es muy simple ya que solamente se debe agregar la nueva columna a la única tabla presente.

B. Desventajas de esta solución

1. La tabla siempre contiene tuplas con algunas columnas vacías. Si la tupla corresponde a un empleado (Clase Employee) todos los campos correspondientes a la clase Customer estarán vacíos, lo cual es una pérdida de espacio al menos.

2. Se torna difícil cuando existe el mismo atributo en dos clases de la misma jerarquía, pero con diferente semántica o rango de valores posibles.

3. Es necesario incorporar una nueva columna a la tabla que especifique a qué clase de la jerarquía pertenece la tupla que se está leyendo[23].

[23] En la figura 13 se muestra esta nueva columna con el nombre objectType.

4. Cada vez que se quiera hacer una consulta que recorra las instancias de una clase concreta (por ejemplo Employee) se estarán recorriendo también las instancias de las demás clases.

5. Al realizar cambios en la estructura de una clase siempre se termina impactando en el mapeo de todas las clases, ya que todas las clases comparten la misma tabla destino.

Mapeo con una tabla por clase

En esta alternativa cada una de las clases recibe una tabla que albergará su información, incluso para el caso de las clases abstractas. Siguiendo con el ejemplo de la jerarquía de Personas, el mapeo resultaría en las siguientes tablas:

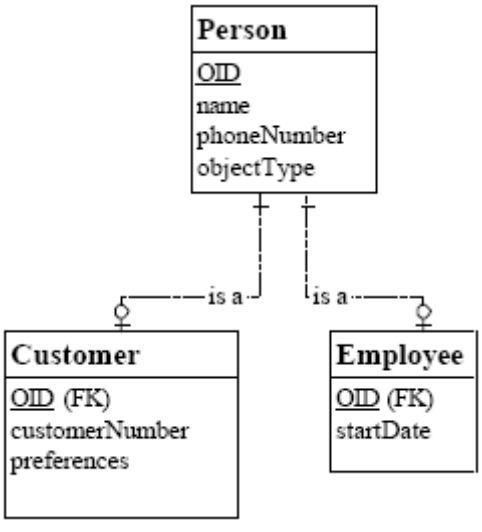

Figura 14: Mapeo de todas las clases con su correspondiente tabla

Como puede observarse, la tabla Person es la que mantiene la identificación de las instancias, y luego cada tabla "concreta" (es decir, cada tabla perteneciente a una clase concreta) tiene un identificador como clave foránea (FK) a la tabla Person, aumentando la información de la tabla Person con los datos propios.

A. Ventajas de esta solución

1. Desde el punto de vista de la orientación a objetos, esta alternativa es la que más se asemeja. Se tiene una tabla que representa la clase abstracta, en la que se definen los atributos comunes a todas las subclases; y luego tablas para las clases concretas que "apuntan" a la tabla de la clase abstracta.

2. Cuando se realiza un cambio que debe propagarse a cada clase concreta es necesario alterar solamente la clase abstracta. El resto de las tablas no debe ser modificado. En este sentido es similar a la alternativa anterior.

3. Cuando se requiere consultar el conjunto de instancias de una clase en particular, simplemente basta con apuntar la consulta a la tabla asociada sin tener que recorrer inútilmente el resto de las instancias (siempre y cuando no se requieran datos que están en la tabla Person).

4. Si hay un atributo con el mismo nombre en dos o más clases, cada una lo puede mapear como más le convenga.

5. Para aquellas consultas que deban apuntar a todas las instancias de la clase Person (recordar que todas las instancias de las clases Employee y Customer son en definitiva también instancias de la clase Person) basta con dirigir la consulta a una sola tabla (Person). Esto suponiendo que dicha tabla contiene toda la información requerida.

B. Desventajas de esta solución

1. Quizás la mayor desventaja de esta solución de mapeo resida en la necesidad de realizar siempre joins para poder recuperar toda la información que compone a una instancia. Esto se debe a que la parte común a todas las instancias se encuentra en la tabla Person y la parte correspondiente a la Clase Customer está en la tabla Customer.

Mapeo de las clases concretas únicamente

En esta alternativa se mapean solamente las clases concretas, por lo que es necesario "mover" todos los atributos presentes en la clase abstracta a cada una de sus subclases concretas. Con este esquema, las tablas resultantes serían las que se muestran en la figura 15

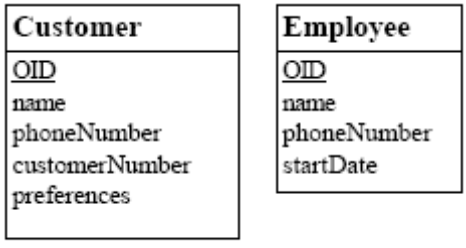

Figura 15: Tablas resultantes del mapeo de clases concretas únicamente

Como puede observarse, los atributos *name* y *phoneNumber* (definidos en la clase abstracta Person) ahora aparecen en las tablas Customer y Employee.

A. Ventajas de esta solución

1. La principal ventaja de esta propuesta es que de ahora en más para recuperar toda la información de una instancia basta con recuperar una tupla de la tabla que corresponda. No es necesario ejecutar **joins** entre varias tablas para "armar" nuevamente una instancia.

2. Otra de las ventajas es que los conjuntos de instancias están separados. Esto permite recorrer solamente alguno de los conjuntos al tratar de recuperar una instancia (o conjunto de instancias) determinadas. Por ejemplo, si se quiere recuperar un cliente, no es necesario recorrer los empleados (en el caso del mapeo de toda la jerarquía a una sola tabla esto suele suceder, sobre todo si no se cuenta con índices apropiadamente definidos).

B. Desventajas de esta solución

1. Cuando se requiere devolver el “extent” de una clase (es decir, el conjunto de todas sus instancias), y dicha clase es la que fue “dividida” (en nuestro ejemplo sería la clase Person) se debe ejecutar una de las operaciones más caras del SQL: UNION. Entonces para recuperar todas las personas habría que realizar la siguiente consulta

   *select * from Customer*

   UNION

   *select * from Employee*

2. Otra desventaja tiene que ver con la estructura repetida en varias tablas. Supongamos el siguiente caso: se desea agregar un nuevo atributo a la clase abstracta Person. En este caso se debe obviamente agregar el atributo en la clase y luego recordar en qué tablas se repartió la información de esta clase para agregar tantas columnas como sea necesario. Para ser más concretos, habría que recordar que existen dos tablas (Customer y Employee) que representan a las instancias de las clases Customer y Employee respectivamente, y que ambas deben contener ahora un nuevo atributo. Con 2 o más tablas no representa un gran problema, pero en diseños de sistemas grandes en donde se cuenta con 2000 tablas o más esto se vuelve un problema grave (sobre todo considerando que la documentación no es precisamente el foco de nuestra atención). Para el caso del borrado de un atributo sucede lo mismo.

Mapeo de asociaciones, composiciones y agregaciones

Una asociación no es más que un vínculo existente entre dos instancias. Por otro lado, la manera de vincular tuplas de una base de datos relacional es a través de relaciones, por lo tanto para mapear asociaciones se deberán utilizar relaciones (1 a 1, 1 a n, etc.).

Antes de comenzar con el análisis de cada una de las posibles situaciones al mapear asociaciones conviene refrescar los conceptos de agregación y composición del paradigma orientado a objetos:

- Agregación: es una asociación entre dos instancias, en la que una de las instancias forma parte de la otra. No existe relación entre los tiempos de vida de cada una de las instancias.
- Composición: es una asociación "fuerte" entre dos instancias, en donde se establece la relación "es parte de" y por lo tanto se vinculan los tiempos de vida de ambas instancias. En otras palabras, al desaparecer el objeto "contenedor" debe desaparecer el objeto "parte".

¿Qué diferencias entonces aparecen al mapear, mediante relaciones, una agregación o una composición? La respuesta es, en realidad, ninguna o casi ninguna. "Ninguna" en el sentido de que si uno mira únicamente las tablas y relaciones resultantes, no se puede distinguir si se trata de una agregación o de una composición. "Casi ninguna" porque puede ser que aparezcan otros elementos de las bases de datos relacionales, como Stored Procedures o Triggers que se disparen en cascada para eliminar una tupla (parte) cuando se elimina otra tupla (contenedor).

Asociaciones 1 a 1

Este tipo de asociaciones se mapea directamente agregando en una tabla una columna más que mantendrá la clave primaria de tupla de la otra tabla referenciada. En el caso de que se trate de una asociación bidireccional, ambas tablas deberán agregar una columna

extra para referenciar la clave primaria de la otra entidad. Es el caso más simple ya que no es necesario crear una tabla intermedia.

Asociaciones 1 a n

Este tipo de asociaciones generalmente está relacionado con el mapeo de las colecciones de objetos. En este caso se debe seguir la misma idea que al pasar a tablas un diseño relacional en donde se tienen relaciones 1 a n, es decir "pegando" del lado de n la clave primaria del lado de 1. Para ser más explícitos, si tenemos una clase *Persona*, que tiene varios Teléfonos (en otras palabras la clase Persona tiene una colección de teléfonos), ambas clases serán mapeadas a sendas tablas *Tabla_Persona* y *Tabla_Teléfono* respectivamente. Adicionalmente a la tabla *Tabla_Teléfono* se debe agregar una columna extra para referenciar la clave primaria de *Tabla_Persona*, de modo de poder determinar para cada teléfono quién es la persona dueña del mismo. Nuevamente no ha sido necesaria la creación de una tercera tabla.

Asociaciones n a n

Este caso es bastante parecido al anterior, con la salvedad de que ahora no basta con agregar columnas a las tablas resultantes del mapeo, sino que se debe crear una tercera tabla que mantenga las relaciones entre las tuplas de la relación "n a n".

Siguiendo con el ejemplo, si tenemos la clase *Persona* y la clase *Cuenta* (en donde una persona puede tener varias cuentas, y a su vez una cuenta pueda pertenecer a varias personas) en mapeo resultante podría ser:

Tabla_Persona (**persona_pk**,….)

Tabla_Cuenta (**cuenta_pk**,….)

y finalmente la relación que une a cada persona con sus cuentas y viceversa,

Tabla_Persona_Cuenta(**persona_pk**, **cuenta_pk**, …).

Un punto adicional a discutir es el relacionado con las diferencias que se deben tener en cuenta al mapear colecciones ordenadas. En este caso, al recuperar los elementos de una colección debemos poder recuperarlos exactamente en el mismo orden en el que se encontraban al momento de persistirlos.

¿Cómo se puede lograr mantener el orden? En realidad es sumamente sencillo. Basta con agregar una nueva columna que mantenga el "orden" de la tupla dentro de la colección de un objeto (representado a través de su clave primaria). En este punto hay detalles que pueden hacer bastante más complejo este tema: ¿qué sucede al eliminar elementos de la colección ordenada? y ¿cómo afecta la presencia de un orden a la performance? Ambos puntos serán comentados con mayor profundidad en los próximos capítulos.

Mapeo del identificador

El OID es el identificador de una instancia, generalmente oculto al desarrollador, que permite identificar unívocamente al objeto. Si la información de un objeto será almacenada en una tupla, entonces es necesario definir alguna manera de identificar unívocamente también a dicha tupla. Esto puede lograrse mediante la definición de una clave primaria.

Si bien parecen ser exactamente los mismos conceptos, hay una diferencia importante entre OID y clave primaria. En la teoría de bases de datos relacionales se suele proponer que la clave primaria esté compuesta por valores del dominio, en cambio los OIDs son administrados enteramente por el ambiente, haciendo muchas veces imposible siquiera el poder consultarlo.

Esto último plantea el primer problema importante al momento de mapear un objeto a una tupla. Supongamos el siguiente caso: existe una instancia de la clase *Persona*, cuya información se almacena en la tabla *TPersona*. Al modificar la información en la instancia y persistir dichos cambios, ¿cómo se hace para determinar cuál de todas las tuplas de la tabla que corresponda es la que representa a la instancia que estoy modificando (recordar que desde el mundo de objetos no suele tenerse acceso al OID)?

La solución más simple que se suele adoptar es agregar un nuevo atributo a la clase. Dicho atributo tendrá como finalidad almacenar el valor de la clave primaria, de modo que cuando se desee persistir el objeto se podrá acceder rápidamente a la clave que identifica la tupla asociada. Esta solución es simple y funciona. Pero no es del todo elegante desde el punto de vista de objetos ya que terminamos por romper una de las primeras características deseables del mapeo: la transparencia. A partir de ahora todo diseñador OO "sabe" explícitamente que las instancias de una clase se está persistiendo en una base de datos relacional.

Obviamente existen otras alternativas para solucionar este problema pero todas, en mayor o menor medida, terminan por romper la transparencia, algunas agregando incluso bastante más complejidad.

Hibernate

Unos años atrás cuando uno se proponía desarrollar una aplicación orientada a objetos con persistencia relacional, en forma indirecta terminaba desarrollando software para realizar el mapeo entre ambos paradigmas. No sólo es tedioso y complicado desarrollar este tipo de software, sino que además probablemente uno podía hacer realmente poco reuso entre aplicaciones distintas, por lo que se debía volver a desarrollar dicha capa de mapeo una y otra vez.

Adicionalmente, como el objetivo de la aplicación es resolver problemas de un dominio determinado, seguramente los desarrolladores conozcan más dicho dominio que el dominio de la persistencia. Por esta razón, es evidente que las soluciones de persistencia "caseras" no eran óptimas, y mucho menos estándares.

Todo eso comenzó a cambiar a fines del año 2001 con la publicación en SourceForge de un proyecto open-source denominado Hibernate, el cual a través de los años se convertiría prácticamente en el estándar de facto para el mapeo de objetos a bases de datos relacionales.

Hibernate ha sido tan importante, primero en el mundo Java, que incluso ha sido portado a otras plataformas como .Net con su implementación open-source NHibernate.

Arquitectura

La arquitectura de este producto ha ido evolucionando a lo largo de los años, pero básicamente sigue respetando la división inicial en tres grandes componentes:

- Administración de las conexiones: Hibernate administra las conexiones a las bases de datos de manera de optimizar este recurso tan costoso.
- Administración de las transacciones: el mantenimiento de las propiedades ACID es primordial para el correcto funcionamiento del sistema, por lo que Hibernate no lo deja librado al usuario sino que es directamente responsable de administrar las transacciones.
- Mapeador: este componente es el encargado de insertar, actualizar, borrar y recuperar la información persistida. Se apoya en los otros componentes así como en otros recursos que deben ser definidos por el usuario.

La figura 16 muestra un esquema simplificado de la arquitectura de Hibernate

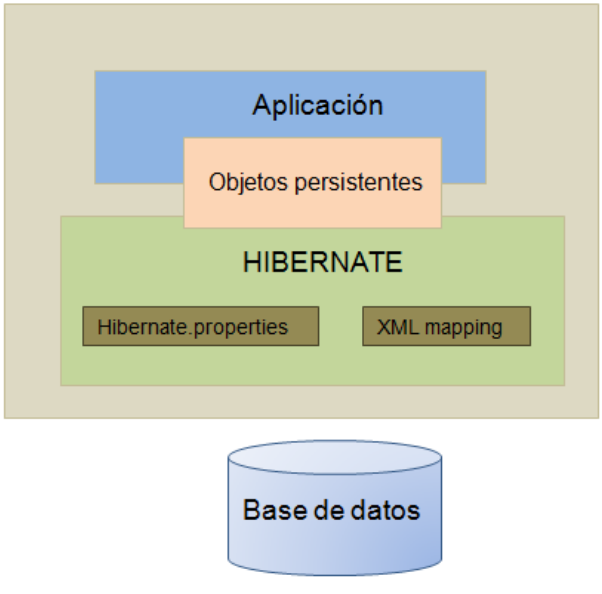

Figura 16: Arquitectura de Hibernate

Como puede observarse a partir de la figura anterior, no hay más interacción directamente entre la aplicación y la base de datos. La aplicación sólo trabaja con los objetos persistentes, los cuales al estar administrados por Hibernate, son persistidos en forma "cuasi-automática".

Forma de trabajo

Para trabajar con Hibernate es necesario realizar las siguientes actividades:

- Crear las clases correspondientes al modelo. Este paso no tiene demasiadas variantes respecto de lo que se haría en un caso sin persistencia, siempre y cuando se tomen los recaudos suficientes. Por recaudos necesarios nos referimos tanto a cuestiones obvias a estas alturas, como no escribir SQL en las clases del dominio; como a cuestiones más bien sutiles, como por ejemplo el uso de annotations. Estas últimas, sin bien en algunos casos pueden resultar de utilidad, al utilizarlas relacionadas con la persistencia traen más desventajas que ventajas (al final de este capítulo se encuentra una breve discusión sobre este punto). A diferencia de otras alternativas de persistencia, con Hibernate no es necesario post-compilar las clases o aplicarles ningún otro proceso.

- Crear archivos de mapeo: para que Hibernate conozca cuales son las tablas a las que se debe enviar la información de cada clase, qué estrategia de mapeo de jerarquía se decidió utilizar, y toda otra información oportuna, se deben crear archivos de mapeo. Estos archivos contendrán para cada clase (o jerarquía de clases) toda la información requerida para poder persistir y luego recuperar los datos. Estos archivos mantienen el nombre de la clase pero su extensión es ".hbm.xml". Entonces si estamos mapeando la clase Car, tendremos el archivo de mapeo Car.hbm.xml. Estos archivos suelen ser ubicados en los mismos directorios que las clases que mapean, aunque con configuración este punto puede ser alterado.

- Configurar las librerías de Hibernate. Aunque resulte obvio este punto, hay que incluirlo en la lista ya que muchas veces los errores que se levantan a raíz de la utilización de este producto tienen que ver con la falta de librerías para su correcto funcionamiento. Para facilidad del usuario, en estos momentos las

librerías de Hibernate están divididas teniendo en cuenta si se tratan de extensiones o es parte del núcleo básico requerido.

- Configurar Hibernate. Además de los mapeos propiamente dichos hay que configurar Hibernate con información del esquema de base de datos al cual debe acceder, con qué protocolo de SQL debe hacerlo, si debe utilizar caché o no, etc. Todos estos elementos son configurados en un archivo denominado hibernate.cfg.xml, el cual es leído una única vez al levantar la aplicación para optimizar el proceso de arranque y configuración.

- Abrir una sesión de trabajo y trabajar con los objetos. Finalmente, el último paso que resta es abrir una conexión a la base de datos a través de Hibernate, operar con los objetos del dominio y al cerrar la sesión de trabajo, si todo está configurado correctamente, los cambios deberían verse reflejados en la base de datos.

Principales componentes

En esta sección se comentarán brevemente los principales componentes de Hibernate más utilizados.

- SessionFactory: es una caché (inmutable) de mapeos compilados para una base de datos. Además su función es permitir la creación de sesiones de trabajo.

- Session: representa una "conversación" entre la aplicación y el almacenamiento persistente. Permite persistir y recuperar objetos y crear transacciones. Además mantiene una caché de objetos persistentes, usada cuando se navega el grafo de objetos o para buscar objetos por identificador.

  Dentro del protocolo de Session tenemos un método muy relevante:

  public Serializable **save**(Object object) throws HibernateException

  Este método es la base para implementar un mecanismo de persistencia transparente ya que para almacenar un objeto solamente es necesario invocar a este método pasándole el objeto que se desea persistir.

Ahora bien, si tenemos en cuenta que Hibernate provee **persistencia por alcance**, la pregunta es ¿cuántas veces es necesario invocar a este método dentro de la aplicación?

Aunque suene extraño, la respuesta es 0 (cero). ¿Cómo es esto de que se puede persistir información a través de Hibernate invocando exactamente cero veces el método save()? Bien, para responder a esta pregunta podemos analizar escenarios y ver que todos reducen siempre al mismo caso base.

Escenario 1: supongamos que tenemos una aplicación que invoca n veces al método **save()**. En este caso hay que notar que Hibernate realizará un análisis de su la instancia que se está pasando como parámetro es nueva o si en cambio ya está persistida y sólo hay que actualizarla. En cualquier caso se guardará la información de esta instancia, pero además Hibernate debe asegurarse que todos los objetos que "relacionados" con dicha instancia también estén actualizados. Por ejemplo, si tenemos un Banco y estamos agregando un nuevo Cliente a dicho banco (llamando al **save()** con el cliente), hay que notar que Hibernate revisará la instancia de Banco para ver si no debe actualizarla también. Como conclusión podemos ver que aún cuando se llame al **save()** con un objeto determinado, Hibernate debe revisar los objetos relacionados para actualizarlos posiblemente.

Escenario 2: ¿qué pasa si en vez de llamar directamente al **save()** utilizamos persistencia por alcance?. En este caso deberíamos recuperar la instancia del Banco y ejecutar el método **add()** de la colección de clientes del Banco. Al realizar esto, el banco quedará marcado como modificado y Hibernate persistirá todos los cambios automáticamente (además de insertar al nuevo cliente). En este escenario podríamos considerar que solamente es necesario llamar una sola vez al método **save()** (para persistir la primer instancia del Banco).

Escenario 3: si bien es cierto y muy probable el escenario 2, todavía tenemos un **save()**. Si ahora consideramos que el Banco y otros objetos son objetos requeridos para la correcta ejecución de la aplicación, uno puede darse cuenta de que ya deberían estar creados antes de la primera ejecución de la aplicación. Si seguimos con este razonamiento pronto nos daremos cuenta de que la aplicación

puede "asumir" que el Banco ya existe y no necesitará nunca llamar al método **save()**[24]. Todo esto se debe nuevamente a la posibilidad que da la **persistencia por alcance**.

- Transaction: define una unidad de trabajo atómica.

Ahora que conocemos los elementos más importantes podemos ver un pseudo-código que muestra la utilización de Hibernate. La figura 17 muestra dicho código

```
Session sess = factory.openSession();
Transaction tx;
try {
    tx = sess.beginTransaction();
    //do some work
    ...
    tx.commit();
}
catch (Exception e) {
    if (tx!=null) tx.rollback();
    throw e;
}
finally {
    sess.close();
}
```

Figura 17: Pseudo-código de utilización de Hibernate[25]

Como puede observarse en la figura 17, la forma de trabajo es relativamente simple

1. Se crea una nueva session a través de una instancia de SessionFactory (en la figura se denomina *factory*).

2. Para comenzar a trabajar con los objetos se abre una transacción a partir de la session.

[24] El banco y los demás objetos requeridos pueden ser creados a través de scripts de SQL directamente en la base de datos o a través de instaladores.

[25] Este pseudo-código figura en la API de la clase Session provista por Hibernate.

3. Al finalizar el trabajo se cierra la transacción (*commit()*) y si todo funciona bien los cambios se persisten a la base de datos.

En la siguiente sección se muestra un ejemplo simple pero completo para ejemplificar el uso básico de Hibernate.

Ejemplo simple de uso

A continuación se presentan todos los artefactos relacionados con un ejemplo simple de utilización de Hibernate[26]. Si bien hay partes que no fueron explicadas explícitamente en las secciones anteriores, son relativamente simples los conceptos por lo que el lector no debería tener problemas en comprenderlos.

La figura 18 presenta el diagrama de clases que se desea persistir.

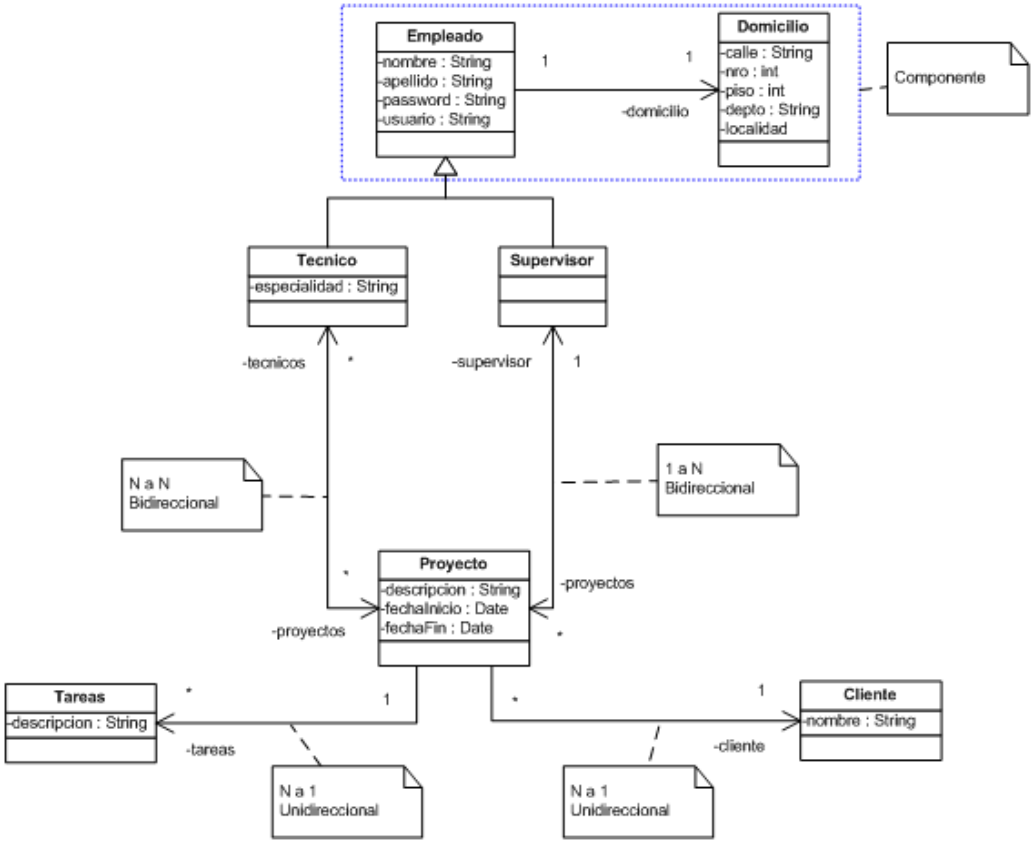

Figura 18: Diagrama de clases del ejemplo.

[26] El código de las clases del modelo no se muestra aquí ya que no tienen ninguna complejidad adicional. El proyecto en SourceForge que acompaña este libro contiene este código en caso de que el Lector quiera obtenerlo.

Como puede observarse en la figura anterior, se trata de un modelo de objetos simple y pequeño. El único detalle que merece la atención es la declaración del domicilio de un empleado como un "componente", lo que permite recuperar al domicilio en el mismo momento que se recupera al empleado relacionado.

En la siguiente figura puede verse el archivo de configuración de Hibernate, hibernate.cfg.xml, en el que se pueden ver todos los elementos que se deben configurar para poder utilizar apropiadamente este mapeador. Entre los parámetros que se deben configurar tenemos el driver de conexión a la base de datos, usuario y clave para la misma y adicionalmente se deben definir todos los archivos de mapeo que se deben importar.

Usualmente durante la etapa de desarrollo es habitual configurar el parámetro *"show_sql"* para poder ver en la consola las sentencias sql que genera automáticamente Hibernate.

```
<?xml version='1.0' encoding='utf-8'?>

<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<hibernate-configuration>

        <session-factory>
          <property name="dialect">org.hibernate.dialect.HSQLDialect</property>
          <property name="connection.driver_class">org.hsqldb.jdbcDriver</property>
          <property name="connection.username">example</property>
          <property name="connection.password"></property>
          <property name="connection.url">jdbc:hsqldb:file:data/DB</property>
          <property name="show_sql">true</property>

          <mapping resource="hibernate/Empleado.hbm.xml" />
          <mapping resource="hibernate/Cliente.hbm.xml" />
          <mapping resource="hibernate/Tarea.hbm.xml" />
          <mapping resource="hibernate/Proyecto.hbm.xml" />
        </session-factory>
</hibernate-configuration>
```

Figura 19: Archivo de configuración de Hibernate.

La figura 20 muestra el archivo de mapeo correspondiente a la clase Tarea, Tarea.hbm.xml. Como todo archivo XML debe ser bien formado y respetar las etiquetas de Hibernate adecuadamente. En la siguiente figura puede apreciarse como se configura

una columna para la clave primaria (*oid*) además de otras columnas para el nombre y la descripción (en el caso de estas dos últimas columnas, se tomará el nombre del atributo presente en la clase para el nombre de la columna en la tabla a falta de una definición de dicho nombre en el archivo de mapeo).

```
<?xml version="1.0"?>

<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"

"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">

<hibernate-mapping package="modelo">

        <class name="Tarea">

                <id name="oid" type="long" column="oid_tarea" unsaved-value="0">
                        <generator class="increment" />
                </id>

                <property name="nombre" />
                <property name="descripcion" />

        </class>
</hibernate-mapping>
```

Figura 20: Archivo de mapeo de la clase Tarea.

En la figura 21 tenemos el archivo de mapeo de la clase Empleado, Empleado.hbm.xml.

En este archivo tenemos no sólo el mapeo correspondiente a la clase Empleado, la cual es abstracta, sino que también hemos puesto las definiciones de los mapeos de las subclases Supervisor y Técnico.

Otro punto a notar es la definición del domicilio como *componente* del empleado. En Hibernate esto significa que siempre que se recupere un empleado se recuperará también su domicilio sin necesidad de ejecutar una consulta adicional. También hay que notar que un domicilio no puede recuperarse sólo, sino que siempre debe ser accedido a través de su empleado ya que no cuenta con un identificador propio.

Finalmente, se puede ver como algunos atributos generales son mapeados en la clase Empleado (como *nombre* y *apellido*); mientras que otros atributos propios del supervisor son declarados únicamente para esta clase (colección *proyectos*) y en forma similar para los técnicos (atributo *especialidad*).

```
<?xml version="1.0"?>

<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">


<hibernate-mapping package="modelo">
	<class name="Empleado">
		<id name="oid" type="long" column="oid_empleado" unsaved-value="0">
			<generator class="increment" />
		</id>
		<discriminator column="emp_tipo" type="string" />

		<property name="nombre" />
		<property name="apellido" />
		<property name="password" />
		<property name="usuario" />

		<property name="dni" />

		<component name="domicilio" class="Domicilio">
			<property name="calle" />
			<property name="nro" />
			<property name="piso" />
			<property name="depto" />
			<property name="localidad" type="string" />
		</component>

		<subclass name="Supervisor" discriminator-value="S">
			<set name="proyectos" cascade="all">
				<key column="oid_empleado" />
				<one-to-many class="Proyecto" />
			</set>
		</subclass>

		<subclass name="Tecnico"  discriminator-value="T">
			<set name="proyectos" table="tecnicos_proyectos"  cascade="all" >
				<key column="oid_empleado" />
				<many-to-many  column="oid_proyecto" class="Proyecto" />
			</set>
			<property name="especialidad" />
		</subclass>
	</class>
</hibernate-mapping>
```

Figura 21: Mapeo de la clase Empleado.

La figura 22 muestra el mapeo correspondiente a la clase Cliente, Cliente.hbm.xml. A diferencia del mapeo de empleados, este mapeo es sumamente simple.

Hibernate permite utilizar una gran variedad de alternativas para la generación de las claves para las tuplas; en estos ejemplos se ha estado utilizando la estrategia "*increment*" que indica a Hibernate que se haga cargo de generar automáticamente un número incremental para asignar la clave a cada tupla que se inserte.

```
<?xml version="1.0"?>

<!DOCTYPE hibernate-mapping PUBLIC"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">

<hibernate-mapping package="modelo">
        <class name="Cliente">
                <id name="oid" type="long" column="oid_cliente" unsaved-value="0">
                        <generator class="increment" />
                </id>

                <property name="nombre" />
                <property name="dni" />

        </class>

</hibernate-mapping>
```

Figura 22: Mapeo de la clase Cliente.

El mapeo correspondiente a la clase Proyecto, Proyecto.hbm.xml, se puede apreciar en la figura 23.

En esta figura nuevamente contamos con un mapeo mucha más interesante debido a su complejidad, ya que se cuenta con diferentes mapeos:

- de atributos simples (*nombre* y *descripción*);
- colecciones unidireccionales 1 a n con el supervisor (un supervisor supervisa muchos proyectos);
- colecciones bidireccionales (relación entre *proyectos* y *técnicos*);
- colecciones 1 a muchos (colección de *tareas*).

```
<?xml version="1.0"?>

<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping package="modelo">
	<class name="Proyecto">

		<id name="oid" type="long" column="oid_proyecto" unsaved-value="0">
			<generator class="increment" />
		</id>

		<property name="nombre" />
		<property name="descripcion" />
		<property name="fechaInicio" />
		<property name="fechaFin" />

		<many-to-one name="supervisor" column="oid_empleado" not-null="true"
		cascade="all"/>

		<set name="tecnicos" table="tecnicos_proyectos" cascade="all" inverse="true">
			<key column="oid_proyecto" />
			<many-to-many column="oid_empleado" class="Tecnico" />
		</set>

		<set name="tareas" cascade="all">
			<key column="oid_proyecto" />
			<one-to-many class="Tarea" />
		</set>

		<many-to-one name="cliente" column="oid_cliente" not-null="true" cascade="all" />

	</class>
</hibernate-mapping>
```

Figura 23: Mapeo correspondiente a la clase Proyecto.

Finalmente, para ver un extracto de código en el que se utiliza Hibernate para persistir información del modelo tenemos que analizar la siguiente figura en la que se realizan varias actividades, desde configurar Hibernate, crear objetos de dominio para finalmente persistirlos. Es de esperar que en una aplicación productiva todas estas actividades no estén codificadas en un solo lugar, sino que cada una pertenezca a una capa bien identificada.

```
1   Configuration cfg = new Configuration();
2   cfg.configure();
3   new SchemaExport(cfg).create(true, true);
4   SessionFactory sessions = cfg.buildSessionFactory();
5   Session session = sessions.openSession();
6   Domicilio dom1 = new Domicilio("Calle1", 1, 1, "Dpto1", "La Plata");
7   Cliente cli1 = new Cliente("Juan Perez", "1");
8   Supervisor sup1 = new Supervisor("Maria", "Ru", "mr", "1", "1");
9   Tecnico tec = new Tecnico("Juan", "Martin", "ja", "14","5", "elec");
10  sup1.setDomicilio(dom1);
11  Tarea t1 = new Tarea("Analisis", "Tarea1");
12  DateFormat dfm = new SimpleDateFormat("yyyy-MM-dd");
13  Date d = dfm.parse("2006-02-26");
14  Date d2 = dfm.parse("2008-09-26");
15  Proyecto p1 = new Proyecto("Proyecto1", "Descripcion1", d, d2,
16          cli1, sup1);
17  sup1.agregarProyecto(p1);
18  p1.agregarTecnico(tec);
19  tec.agregarProyecto(p1);
20  p1.agregarTarea(t1);
21  Transaction tx = null;
22  try {
23      tx = session.beginTransaction();
24      session.save(p1);
25      tx.commit();
26  } catch (Exception e) {
27      if (tx != null)
28          tx.rollback();
29      session.close();
30  }
31  session.disconnect();
```

Figura 24: Utilización de Hibernate.

Algunas aclaraciones en cuanto a la figura 24:

- En las líneas 1 a 3 se configura Hibernate, indicándole que lea su archivo de configuración y que además cree el esquema requerido en la base de datos.
- En las líneas 4 y 5 se realiza la conexión a la base de datos y se obtiene una sesión de trabajo para poder trabajar con la base de datos.
- Las líneas 6 a 20 crean diferentes instancias del modelo y los vinculan.
- La línea 23 abre una transacción a partir de la sesión de trabajo.
- La única línea requerida para persistir todo el modelo recientemente creado es la línea 24 a través de la invocación del método **save()**. En este caso no contábamos con un modelo persistente por lo que no pudimos hacer uso de la persistencia por alcance en su forma más potente, sin embargo notar que

solamente se está pidiendo explícitamente que se almacene el proyecto y en cascada todos los demás objetos también se persisten.

- La línea 25 realiza el commit de la transacción.
- Las restantes líneas son necesarias para el caso de que ocurra algún error en la persistencia.

Consultas

Esta solución de mapeo brinda 4 maneras de recuperar la información:

- Iteradores: esta es la manera más "orientada a objetos". Si uno recupera un objeto cualquiera, y dicho objeto tiene colaboradores que son colecciones, entonces es posible obtener un iterador y a partir de éste recorrer todos los elementos. Es necesario aclarar que dicho iterador no recupera inmediatamente todos los elementos de la colección, lo que algunas veces significa una mejora en la performance y otras veces puede perjudicarla. En el capítulo dedicado a los patrones de diseño se verá con más detalle este punto.
- SQL: en algunas circunstancias es necesario contar con la posibilidad de ejecutar consultas SQL. A través de la conexión que se obtiene de la sesión de trabajo la aplicación puede enviar todas las sentencias SQL que requiera. Obviamente esta alternativa debe ser utilizada en casos de extrema necesidad y no como método preferido.
- HQL: tomando las mejores características de OQL, Hibernate presenta un lenguaje de consulta "orientado a objetos", con el cual se pueden recuperar los objetos persistidos. Obviamente no todas las características del OQL se han implementado, pero por lo menos las más importantes se encuentran disponibles. Todas las consultas en HQL finalmente son traducidas a SQL para poder ser ejecutadas por el motor de la base de datos relacional. La performance de esta alternativa es muy buena, incluso comparándola con SQL, razón por la cual es la forma preferida de realizar consultas.

Más abajo se encuentran algunas consultas de ejemplo en HQL para demostrar su potencia[27].

1. Listar los nombres de todos los proyectos

```
SELECT p.nombre
FROM modelo.Proyecto p
```

Notar que se debe utilizar el nombre de la clase y no el nombre de la tabla para especificar lo que se quiere consultar.

2. Listar los nombres de los proyectos que tienen como cliente, al cliente cuyo nombre es *"Carlos Rodríguez"*

```
SELECT p.nombre
FROM modelo.Proyecto p
WHERE p.cliente.nombre = 'Carlos Rodriguez'
```

En este caso hay que remarcar que no es posible utilizar métodos, sino que se debe acceder directamente a la estructura.

3. Listar los técnicos que tienen especialidad *"electrónica"*, que están en los proyectos con tarea *"Análisis"* y supervisados por el supervisor con apellido *"Ruiz"*

```
SELECT new list(tecnico.nombre,tecnico.apellido)
FROM modelo.Proyecto p
        join p.tareas as tarea
        join p.tecnicos as tecnico
WHERE tecnico.especialidad = 'electrónica'
        and tarea.nombre = 'Análisis'
        and p.supervisor.apellido='Ruiz'
```

En este ejemplo se devuelve un nuevo tipo de resultado (una lista con dos atributos por cada técnico); además se utiliza la palabra **join** para especificar que se quiere ligar a cada tarea de un proyecto con la variable tarea y en forma similar con la variable técnico para luego verificar condiciones sobre estas variables.

[27] Para las consultas de ejemplo se utilizará el modelo de objetos de la figura 18.

- Criteria: la principal desventaja que tiene HQL es que es dentro del ambiente Java es considerado simplemente como un String, por lo que no es posible detectar errores en su definición. Se debe ejecutar para verificar que esté correcto y que devuelve lo que uno espera. Para solucionar estos problemas, y otros más sutiles, Hibernate también presenta un framework de consulta totalmente orientado a objetos que permite la creación y verificación dinámica de correctitud de la consulta (por lo menos en términos sintácticos). Criteria es traducido a HQL para su ejecución, por lo cual es una de las alternativas más "lentas", además de que no todo lo que se puede escribir en HQL es posible lograrlo con HQL.